

Vježba: CLI alat za analizu studenata

Node.js + moderni JavaScript (fs, path, process, map/filter/reduce)

Cilj

Napraviti **Node.js CLI alat** (program iz komandne linije) koji čita podatke o studentima iz JSON fajla i ispisuje različite statistike. Vježba ne koristi Express ni bazu — fokus je na **ugrađenim Node modulima** (fs, path, process) i **modernom JavaScript-u** (arrow funkcije, destrukturiranje, spread, map/filter/reduce).

Šta dobiješ u starter folderu

- `studenti.json` — niz od 20 studenata sa ocjenama i grupom.
- `analiza.js` — kostur sa TODO komentarima koje treba popuniti.

Šta program treba da radi

Program se pokreće iz komandne linije:

```
node analiza.js <komanda> [argumenti...]
```

Treba podržati sljedeće komande:

Komanda	Šta radi
prosjeck	Ispiše prosjek SVIH ocjena svih studenata.
top <N>	Top N studenata po prosjeku ocjena (sortirano).
po-grupama	Broj studenata po svakoj grupi.
trazi <ime>	Filtrira studente čije ime/prezime sadrži dati string.
export <fajl.json>	Snimi obrađene podatke (sa prosjekom) u JSON fajl.

Primjeri pokretanja

```
node analiza.js prosjek
node analiza.js top 5
node analiza.js po-grupama
node analiza.js trazi Ana
node analiza.js export rezultati.json
```

Koraci u rješavanju

Otvori `analiza.js`. Vidjet ćeš 12 TODO komentara — rješavaj ih redom.

1. Uvezi module

Na vrhu fajla, koristi `require()` da uvezeš `fs` i `path`:

```
const fs = require("fs");
const path = require("path");
```

2. Učitaj JSON podatke

Sastavi putanju i pročitaj fajl:

```
const putanja = path.join(__dirname, "studenti.json");
const studenti = JSON.parse(fs.readFileSync(putanja, "utf8"));
```

3. Pomoćne funkcije

Napravi dvije pomoćne funkcije (savjet: koristi arrow sintaksu):

```
// Prosjek niza brojeva
const prosjekOcjena = (ocjene) =>
  ocjene.length === 0 ? 0 : ocjene.reduce((s, o) => s + o, 0) / ocjene.length;

// Vraća NOVI niz studenata u kojem svaki ima polje "prosjek".
// Koristi spread (...) da ne mutiras originalne objekte.
const saProsjekom = (lista) =>
  lista.map((s) => ({ ...s, prosjek: prosjekOcjena(s.ocjene) }));
```

4. Implementiraj komande

- **prosjek** — koristi `flatMap` da razvuče sve ocjene u jedan niz, pa `reduce`.
- **top N** — `.sort((a, b) => b.prosjek - a.prosjek).slice(0, N)`.
- **po-grupama** — `reduce` sa praznim objektom `{}` kao akumulatorom.
- **trazi** — `filter` sa `.toLowerCase().includes(upit)`.
- **export** — `JSON.stringify(podaci, null, 2) + fs.writeFileSync`.

5. Dispatcher

Na dnu, izvuci komandu i argumente iz `process.argv` i napiši `switch`:

```
const [komanda, ...args] = process.argv.slice(2);

switch (komanda) {
  case "prosjek":      komandaProsjek();           break;
  case "top":          komandaTop(args[0]);        break;
  case "po-grupama":   komandaPoGrupama();         break;
  case "trazi":        komandaTrazi(args[0]);      break;
  case "export":       komandaExport(args[0]);     break;
  default:
    console.log("Dostupne komande: prosjek, top <N>, po-grupama, trazi <ime>, export <putanja>");
}
```

Šta vježbaš

- **fs.readFileSync / writeFileSync** — sinhrono čitanje/pisanje fajlova.
- **path.join + __dirname** — pravljenje putanja koje rade na svim OS-ovima.
- **JSON.parse / JSON.stringify** — konverzija između string-a i JS objekta.
- **process.argv** — argumenti komandne linije.
- **Destrukturiranje sa rest**: `const [k, ...args] = process.argv.slice(2)`.
- **Spread u objektu**: `{ ...s, prosjek }`.
- **Array metode**: `map`, `filter`, `reduce`, `sort`, `slice`, `flatMap`, `find`.
- **Arrow funkcije i template literal** za ispis.

Savjet

Sve potrebne sintakse i koncepte imaš u skripti "**Moderni JavaScript za Node.js**". Ako negdje zapne, otvori odgovarajuće poglavlje (npr. 10 za `map/filter/reduce`, 15 za `fs`, 18 za `process`).

Bonus zadaci (opcionalno)

- Dodaj komandu `prosjek-po-grupama` koja ispisuje prosjek ocjena za svaku grupu.
- Dodaj `--sort=ime` opciju za `top` komandu (sortiranje po imenu umjesto po prosjeku).
- Dodaj komandu `histogram` koja ispiše tekstualni histogram broja studenata po prosjeku (5–6, 6–7, ... 9–10).
- Refactor: prebaci sve na `fs/promises` i `async/await`.

Predaja

- Završen fajl `analiza.js` u vlastitom GitHub repozitoriju.
- Svi testovi (`prosjek`, `top`, `po-grupama`, `trazi`, `export`) moraju raditi.
- U README opisati kako se pokreće svaka komanda.